

See discussions, stats, and author profiles for this publication at: <https://www.researchgate.net/publication/337901222>

A Lightweight Protocol for RFID Authentication

Presentation · December 2019

CITATIONS

2

READS

337

2 authors:



Salahuddin A. Azad

Central Queensland University

31 PUBLICATIONS 397 CITATIONS

SEE PROFILE



Biplob Ray

Central Queensland University

97 PUBLICATIONS 1,210 CITATIONS

SEE PROFILE

A Lightweight Protocol for RFID Authentication

Salahuddin Azad
School of Engineering and Technology
Central Queensland University
Melbourne, Australia
Email: s.azad@cqu.edu.au

Biplob Ray
School of Engineering and Technology
Central Queensland University
Melbourne, Australia
Email: b.ray@cqu.edu.au

Abstract—Radio frequency identification (RFID) technology has many applications such as supply chain management, asset tracking, healthcare and logistics. Since RFID tags and readers communicate through a wireless medium, they are prone to a wide range of attacks. There are a number of measures to safeguard the security of RFID device operations and communications: mutual authentication, confidentiality, indistinguishability, forward security, and desynchronisation resilience. Due to limited computational power and memory, heavy-weight encryption functions cannot be performed in the RFID tags to execute the security protocols. Therefore, RFID security protocols are restricted to light-weight encryption functions such as simple one-way hash function, cyclic redundancy check (CRC), pseudo-random number generator (PRNG) and exclusive-OR (XOR). This paper develops a lightweight secure authentication protocol to mutually authenticate the RFID tag and the reader through an insecure radio communication channel. The protocol assumes that each RFID tag pre-shares a secret key with the reader. The protocol uses two random values to guarantee the freshness of the messages in order to outwit any replay attack. An analysis of the protocol using Scyther verification tool shows that the protocol ensures secure communication between the reader and the RFID tag provided the communication channel between the back-end server and the reader is protected.

Keywords—RFID security, mutual authentication, lightweight encryption

I. INTRODUCTION

Radio frequency identification (RFID) is an important technology that has found its way in many applications such as supply chain management, asset tracking, healthcare and logistics. RFID systems consist of three components – RFID tag, RFID reader, and back-end server [1]. Tags are normally attached to an object and each tag contains a unique identifier. Assets and inventory can be uniquely and automatically identified using RFID tags. An RFID reader receives data from tags through short range radio communication and passes the data to the server. The back-end server stores data about each object and the data can be retrieved using the unique identifier of the RFID tag. The architecture of server based RFID systems is illustrated in Figure 1.

As the medium of communication for RFID tags and readers are wireless, they are prone to a wide

range of attacks. The common attacks that could make RFID communication vulnerable are: *cloning*, *eavesdropping*, *replay attack*, *man-in-the-middle* (MIMT), and *denial-of-service* (DOS) attacks [1],[3]. In cloning attack, the information on a legitimate RFID tag is copied to another tag, which when presented to a reader is accepted as legitimate reader. In eavesdropping attack, the imposter listens to the communication channel to capture the conversation between the tag and the reader. In replay attack, the attacker sends a query to a tag by pretending to be the reader and obtains the response of the tag. The attacker later poses as a legitimate tag by replaying the tag's response captured earlier. In man-in-the-middle attack, an adversary places itself between the reader and the tag, intercepts the messages from both parties and relay those to the parties in original or modified form. The adversary is thus able to deceive both parties by making them think that they are communicating with the intended party. In denial-of-service attack, the adversary prevents the reader and the tag communicating each other by overloading them with too many requests or congesting the communication channel.

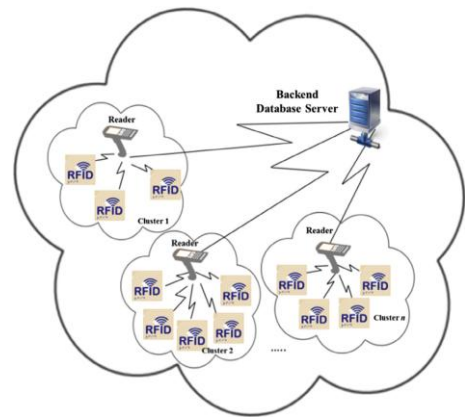


Fig. 1. Architecture of server based RFID systems (adapted from [2]).

There are a number of measures that are adopted to safeguard the security of RFID device operations and communications. These are: *mutual authentication*, *confidentiality*, *indistinguishability*, *forward security*, and *desynchronisation resilience* [4]. Mutual authentication is the process of verifying the identity of reader and tag by each other. Confidentiality means

data should be transmitted via authentication or encryption so that it not revealed to adversaries. Indistinguishability is the property of not being tracked by the transmitted data. To ensure this property, any data sent should be different from what was sent previously. Forward secrecy means the information transmitted in the current session cannot be utilised to reveal the information transmitted in past sessions. Desynchronisation resilience is the property of being resistant to an adversary's attempt to desynchronise a tag and a reader. To be desynchronisation resilient, the shared secret information kept in the tag and reader must be identical.

Due to limited computational power and memory, heavy-weight encryption functions cannot be performed in the RFID tags to carry out security protocols. The RFID security protocols are restricted to light-weight encryption functions such as *simple one-way hash function*, *cyclic redundancy check* (CRC), *pseudo-random number generator* (PRNG), and *exclusive-OR* (XOR) [5],[6]. Some papers attempted to address security issues like confidentiality, and integrity using StenoCipher based solutions [7]. These light-weight encryption functions are not as robust as the classic encryption functions. Therefore, trade-off between security and complexity of RFID encryption protocols remains a technical challenge.

The common properties of lightweight RFID authentication protocols are: (i) use of shared secrets to confirm the identity of both sides; (ii) use of random numbers to prevent replay attacks; (iii) use of message digests to confirm the integrity of the exchanged messages; and (iv) use of XOR operations to prevent adversaries from discovering the shared secrets. The use of random numbers requires the tag to remember all recently used random numbers to prevent the replay attack, which is impractical due to the limitation of tag memory. One way to circumvent this requirement is to get the reader to generate secret random numbers to be used for the next authentication session and shared these numbers secretly with the tag.

The mutual authentication system proposed by [8] is based on lightweight stream cipher called WG-7. The protocol avoids replaying of the authentication session by using two random numbers generated by the reader and the tag. The random number is hidden by an XOR operation with tag ID. To mutually authenticate each other, the reader and the tag exchanges message digests using WG-7 cipher. The cipher operation is initialised by calculating the XOR output of the random numbers and the key. The first and second 80 bits of WG-7 output key stream are used as message digests alternatively. The weakness of the protocol lies in the XOR output of the random numbers. If an adversary can acquire previous communication messages, it can inject additional XOR operations to replace the new random number with the old one from

the previous message digest. Safkhani et al. [9] fixed the problem by replacing the XOR operations in the message digest with concatenation operation.

The mutual authentication protocol developed by Shen et al. [10], named ARAP, is similar to the abovementioned protocol where authentication sessions are randomized using two random values. The protocol changes its pseudonym after each run of the protocol. It is selected from a collision free set. The message digest is calculated from the XOR outputs of the random numbers and pseudonym. Again this XOR operation makes the authentication protocol vulnerable like the previous one. An adversary can acquire previous communication messages and insert additional XOR operations to replace the new random number from with the old one in the message digest. Safkhani et al. [9] also fixed the vulnerability of the protocol by the same way as the previous one.

In the hash based RFID tag mutual authentication scheme proposed by Cho and Kim [11], the tag uses two messages for authentication. The first message is generated using light weight modular arithmetic and XOR operations to hide the random number and achieve efficiency. The second message uses hash operation to provide authentication and enhance security. To ensure the authenticity of the messages, the reader generates two new secret numbers and sends them to the tag to be stored for later authentication message exchanges. If any intruder tries to replay any old authentication messages, the authentication procedure will fail as the server will try to recover the random number with latest secret values shared between the tag and the server. This authentication procedure requires both the tag and server to remember the latest secret values.

The HMAC-based RFID mutual authentication protocol developed by Jung and Jung [3] used physical unclonable function (PUF) feature to avoid cloning of the RFID tags. Like the previous protocol, it updates the challenge response pair after each successful authentication session to maintain the freshness of the messages. It also uses timestamps which safeguards the protocol from replay attack in case of update failure at the server.

The IoT authentication protocol proposed by Esfahani et al. [12] uses preshared key to authenticate the smart IoT sensor and the router. Random numbers are used to ensure the freshness of the messages so as to prevent replay attack, while message digests are used to confirm the integrity of the exchanged messages. As a shared secret key is used to help confirm the identity of both parties, the protocol uses XOR and hash functions to prevent disclosure of the shared secret key. The session key is calculated as the message digest of the concatenation of the two random numbers. The protocol is suitable for IoT environment but may not be suitable for simple RFID authentication

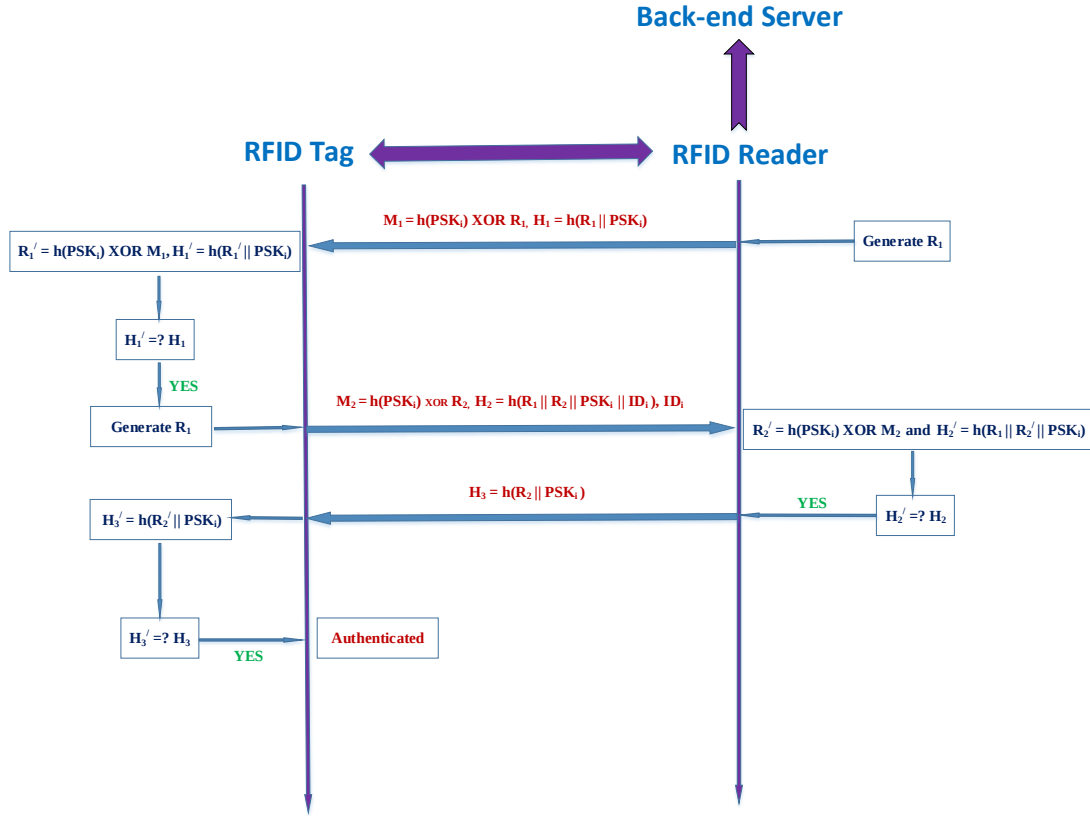


Fig. 2. Proposed mutual authentication protocol between RFID tag and reader.

as it involves a significant number of mathematical and logical operations as well as too many message exchanges.

This paper develops an RFID authentication protocol for secure communication between RFID tags and reader. The paper assumes that the communication channel between the reader and the back-end server is secure. Thus the primary focus of this paper is to develop a lightweight secure authentication protocol to authenticate the tag and the reader through an insecure radio communication channel. The rest of the paper is organised as follows. Section II describes the proposed protocol. Section III discusses the security claim verification process and Section IV discusses the experimental setup for security claim verification. Section V presents the results of the simulation of the proposed protocol. The analysis in Section VI verifies the correctness of the proposed protocol based on the simulation results. Finally, Section VII concludes the paper.

II. THE PROPOSED PROTOCOL

The protocol assumes that there will be a secret pre-shared key, PSK_i between the reader and each RFID tag, TAG_i . This secret pre-shared key is to be used to mutually authenticate the reader and the RFID tag.

Two random numbers R_1 and R_2 are used to maintain the freshness of the messages exchanged between the reader and the tag.

In the first step, the reader generates a random number R_1 , and calculate $M_1 = h(PSK_i) \oplus R_1$ and $H_1 = h(R_1 || PSK_i)$. The hash, H_1 is to ensure the integrity of R_1 and PSK_i . The value of R_1 is masked by $h(PSK_i)$.

In the second step, the reader sends M_1 and H_1 to tag TAG_i .

In the third step, TAG_i calculates $R_1' = h(PSK_i) \oplus M_1$ and $H_1' = h(R_1' || PSK_i)$. If H_1' equals H_1 , then it proves the integrity of R_1 and PSK_i . So TAG_i performs the next step. Otherwise, the message integrity is lost, so the authentication session is abandoned. However, H_1' being equal to H_1 does not necessarily confirm that the message is from the reader as an imposter can copy any previous message and replay to TAG_i . The message replay is resolved in Step 9.

In the fourth step, TAG_i generates another random number R_2 , and calculates $M_2 = h(PSK_i) \oplus R_2$ and $H_2 = h(R_1 || R_2 || PSK_i || ID_i)$. The hash, H_2 is to ensure the integrity of R_1 , R_2 , PSK_i and the tag

identification, ID_i . Again the value of R_2 is masked by $h(PSK_i)$.

In the fifth step, TAG_i sends M_2 , H_2 and ID_i to the reader.

In the sixth step, the reader calculates $R'_2 = h(PSK_i) \oplus M_2$ and $H'_2 = h(R_1 || R'_2 || PSK_i || ID_i)$. If H'_2 equals H_2 , then it proves the integrity of R_1 , R_2 , PSK_i and ID_i . This confirms that the message is indeed from TAG_i . This is because if an imposter tries to replay any previously captured message, the value of R_1 in the captured message would not satisfy the condition, $H'_2 = H_2$.

In the seventh step, the reader calculates $H_3 = h(R_2 || PSK_i)$.

In the eighth step, the reader sends H_3 to TAG_i .

In the ninth step, TAG_i calculates $H'_3 = h(R_2 || PSK_i)$. If H'_3 is equal to H_3 , then it proves that this message is from a legitimate owner of PSK_i . This step confirms that the party sending this message is indeed the reader, not an imposter replaying old messages.

The steps involved and messages exchanged in the proposed mutual authentication protocol are depicted in Figure 2.

Table 1. Settings for Scyther

Verification parameters	Scyther Parameters
Maximum number of Runs	5
Matching Types	Find basic type flaws
Search pruning	Find best attack
Maximum number of patterns per claim	10

III. SECURITY CLAIM VERIFICATION

The security claim verification is a process to assess communication protocol's security claims, identify possible vulnerabilities, and devise solutions to address those vulnerabilities. Traditionally, the security claims of a secure communication protocol are verified by mathematically modelling using BAN or GNY logics [13]. But due to the difficulty of plotting a large number of mathematical/logical statements, increased complexity of communication protocols, and intricacy of plotting multi-protocol situations [14], automated security claim verification techniques like AVISPA and Scyther are getting increasingly popular. This is because automated security claim verification reduces human error, increase robustness and provide visual illustration of various outcomes including attack graphs and information flows. In this paper, an automated security claim verification tool, Scyther [14] is used to identify the vulnerabilities in the proposed authentication

protocol and to adjust the protocol's data flow to address the vulnerabilities so that attackers cannot exploit it to perpetrate attacks.

Table 2. Role characterization for correctness and completeness

Roles	Status	Patterns
C (Back-end Server)	Reachable – ok - verified	At least 1 trace patterns
G (Reader)	Reachable – ok - verified	At least 1 trace patterns
S (RFID Tag)	Reachable – ok - verified	At least 1 trace patterns

IV. EXPERIMENTAL DETAILS

Scyther verification tool was setup in an Ubuntu virtual machine in VMware Player that has version 4.0.0 (build-471780). The Scyther verification tool was setup as per the specifications below.

- Scyther V1.0 [15] was used for this experiment. The parameters that are used in the experiment are shown in Table 1. As presented in the table, "Maximum number of runs" parameter was set to 10 for the verification. The protocol changes its secret values in every run and choose them randomly. Therefore, higher number of protocol runs will be as secure as 10 runs.
- In our verification, it was chosen to check against best possible attacks by setting "Find best attacks" option as shown in Table 1.
- The proposed protocol, adversary model and claims were modelled using Security Protocol Description Language (SPDL) in Scyther [13]. All micro details, such as XOR function, hash and concatenation function, dataflow of the proposed protocol were used to verify the security claims.
- In addition to Scyther automatic claim verifications, several security properties such as secrecy of private values were manually claimed using claim () function of SPDL.

Table 3. Auto and manual security claim verification results

Claims	Role involved	Status
Weakagree	G and S	No attacks
Alive	G and S	No attacks
Nisynch	G and S	No attacks
Niagree	G and S	No attacks
Secret (psk, r1,r2)	G and S	No attacks within bounds
Secret (M1,H1)	G and S	No attacks within bounds

G	LAP_IoT,G2	Secret r2	Ok	Verified	No attacks.
	LAP_IoT,G3	Secret r1	Ok	Verified	No attacks.
	LAP_IoT,G4	Secret H3	Ok		No attacks within bounds.
	LAP_IoT,G5	Secret H2	Ok	Verified	No attacks.
	LAP_IoT,G6	Secret M2	Ok	Verified	No attacks.
	LAP_IoT,G7	Secret ID	Ok	Verified	No attacks.
	LAP_IoT,G8	Secret H1	Ok		No attacks within bounds.
	LAP_IoT,G9	Secret M1	Ok		No attacks within bounds.
	LAP_IoT,G10	Alive	Ok	Verified	No attacks.
	LAP_IoT,G11	Weakagree	Ok	Verified	No attacks.
	LAP_IoT,G12	Niagree	Ok	Verified	No attacks.
	LAP_IoT,G13	Nisynch	Ok	Verified	No attacks.
	LAP_IoT,S2	Secret r2	Ok	Verified	No attacks.
S	LAP_IoT,S3	Secret r1	Ok	Verified	No attacks.
	LAP_IoT,S4	Secret H3	Ok	Verified	No attacks.
	LAP_IoT,S5	Secret H2	Ok		No attacks within bounds.
	LAP_IoT,S6	Secret M2	Ok		No attacks within bounds.
	LAP_IoT,S7	Secret ID	Ok	Verified	No attacks.
	LAP_IoT,S8	Secret H1	Ok	Verified	No attacks.

Fig. 3. Automated security claim verification for the proposed protocol.

V. RESULTS

The Scyther verification tool has a database of known attack models which are used to identify the vulnerabilities of an authentication protocol. The Scyther simulation checks roles characterisation for correctness of the protocol's execution. This is to ensure that the roles are performing as claimed in the protocol. As presented in Table 2, all the roles have 'ok' status which means the correctness of the roles is verified. Furthermore, each role has at least 1 trace patterns which will be analysed in analysis section.

The protocols also made automated security claim verification. The results of automated security claim verification are illustrated in Fig. 3. As we can see from Fig. 3, there are no attacks identified for communication between gateway and sensory RFID tags.

Although, the simulation has identified 2 attacks in communication between the back-end server and the reader but, as assumed in the proposed protocol, the communication between the back-end server and the reader is secure as there are many existing security techniques to protect communication between these two. Therefore, any attacks and security issues for communication between the back-end server and the reader was ignored.

The paper made some manual security claim to verify the secrecy claim of pre-shared key and security properties like Weakagree, Alive, Nisynch and

Niagree [14]. As illustrated in Fig. 4 and Table 3, there are no attacks identified by Scyther after analysing the protocols communication flows between reader and the RFID tags.

G	LAP_IoT,G1	Secret psk	Ok	Verified	No attacks.
	LAP_IoT,G2	Alive	Ok	Verified	No attacks.
	LAP_IoT,G3	Weakagree	Ok	Verified	No attacks.
	LAP_IoT,G4	Nisynch	Ok	Verified	No attacks.
	LAP_IoT,G5	Niagree	Ok	Verified	No attacks.
	LAP_IoT,S1	Secret M1	Ok	Verified	No attacks.
	LAP_IoT,S2	Secret H1	Ok	Verified	No attacks.
	LAP_IoT,S3	Secret psk	Ok	Verified	No attacks.
	LAP_IoT,S4	Alive	Ok	Verified	No attacks.
	LAP_IoT,S5	Weakagree	Ok	Verified	No attacks.
	LAP_IoT,S6	Nisynch	Ok	Verified	No attacks.
	LAP_IoT,S7	Niagree	Ok	Verified	No attacks.

Fig. 4. Manual security claim verification for the proposed protocol.

VI. ANALYSIS

The analysis of the communication flows identified in roles characterization of the protocol (as presented in Table 2) shows that there is vulnerability in the communication between the back-end server and the reader. The vulnerability compromises H_1 and H_2 values exchanged between these two as illustrated in Fig. 5.

However, as both H_1 and H_2 is created by multiple session random values, it will be infeasible for an attacker to regenerate same values for the second time. Furthermore, the protocol assumes that the communication channel between the back-end server and the reader is secure (as such communication will be heavily protected with traditional security primitives), the analysis of the communication between these two is out of the scope of the security analysis. Therefore, based on Scyther simulation results, it can be claimed that the proposed lightweight authentication protocol is secure for communication between the reader and RFID tags, if the communication channel between the back-end server and the reader is protected.

VII. CONCLUSION

The paper has proposed a lightweight RFID authentication protocol for secure communication between RFID tags and readers. The protocol uses a simple one-way hash function, a pseudo-random number generator and the exclusive-OR operation to accomplish authentication. Therefore, the protocol can be easily implemented in RFID devices which have limited processing power and memory. The protocol assumes that the communication channel between the reader and the back-end server is secure. Also it


```

graph TD
    Run[Run #1  
any agent C#1 in role C  
C -> any agent C#1  
G -> any agent G#1  
Fresh r1#1, r2#1  
Var H3 -> any H3#1 of type Nonce  
Var H2 -> any H2#1 of type Nonce  
Var ID -> any ID#1 of type Nonce  
Var H1 -> any H1#1 of type Nonce  
Var ID -> any ID#1 of type Nonce] --> Send1[send_1 to G#1  
r1#1]
    Send1 --> Recv4[recv_4 from G#1  
(H1#1, { C#1 } H2#1)]
    Recv4 --> Send5[send_5 to G#1  
{ hash(ConCat(psk, r2#1, ID#1)) } H3#1]
    Send5 --> Claim[claim_C1  
Reachable]
    
    H2[any H2#1 of type Nonce] -.-> Encrypt([encrypt])
    H1[any H1#1 of type Nonce] -.-> Encrypt
    Encrypt -.-> Recv4
  
```

[Id 1] Protocol LAP-IoT, role C, claim type Reachable, cost 23